

# MVP técnico de CommunityLab

---

Alcance resumido para construir una línea de contenido asistida por IA con curaduría humana y persistencia en OCI

Proyecto

CommunityLab Motor inteligente de transformación y distribución para comunidades digitales

**Fecha** 18 de septiembre de 2026

**Base de diseño** Requisitos del Hackathon ONE G10 y decisiones de alcance orientadas a una demostración funcional

# 1 Definición del MVP

CommunityLab recibe interacciones de una comunidad desde JSON o CSV, las normaliza, analiza sentimiento, temas, relevancia y confianza, detecta oportunidades y genera dos activos: un post de LinkedIn y una FAQ. Los borradores pasan por edición y aprobación humana antes de guardar el paquete final en OCI Object Storage.

## Recorrido mínimo demostrable

Cargar lote -> validar y deduplicar -> analizar con LLM -> aplicar reglas -> generar LinkedIn y FAQ -> editar y aprobar -> guardar paquete y manifest en OCI.

## Usuario principal y valor

| Usuario           | Problema                                     | Resultado                               |
|-------------------|----------------------------------------------|-----------------------------------------|
| Community manager | Revisar manualmente conversaciones dispersas | Oportunidades priorizadas con evidencia |
| Marketing         | Redactar contenido repetitivo                | Borradores adaptados al canal           |
| Curador           | Controlar calidad, privacidad y tono         | Cola de edición, aprobación y rechazo   |

# 2 Alcance y límites

| Incluido en el MVP                              | Fuera del MVP                             |
|-------------------------------------------------|-------------------------------------------|
| Carga de JSON y CSV con hasta 500 interacciones | Conectores productivos de Discord o Slack |
| Endpoint POST /api/v1/process                   | Publicación automática en redes           |
| Análisis estructurado con LLM                   | RAG, embeddings o fine tuning             |
| Generación de LinkedIn y FAQ                    | Seis formatos y campañas multimodales     |
| Curaduría Streamlit                             | Portal empresarial con permisos avanzados |
| PostgreSQL para estado y auditoría              | Analítica histórica compleja              |
| OCI para paquetes y artefactos                  | OCI Compute obligatorio                   |
| Docker Compose local                            | Alta disponibilidad y escalado productivo |

## Criterios de éxito

- Procesar al menos 20 interacciones de prueba y mostrar errores por registro.
- Obtener sentimiento, temas, relevancia, confianza y evidencia para cada interacción.
- Detectar al menos tres oportunidades y generar dos tipos de activos.
- Editar, aprobar o rechazar activos desde Streamlit.
- Guardar el paquete aprobado y su manifest con SHA 256 en OCI Object Storage.
- Demostrar tres transformaciones completas y reproducibles.

### 3 Arquitectura mínima

La solución separa presentación, API, dominio, IA y persistencia. El LLM interpreta y redacta; el código controla validaciones, umbrales, estados, idempotencia y almacenamiento.

| Capa            | Tecnología            | Responsabilidad                                 |
|-----------------|-----------------------|-------------------------------------------------|
| Dashboard       | Streamlit             | Carga, resultados y curaduría                   |
| API             | FastAPI y Pydantic v2 | Contratos, validación y fachada del flujo       |
| Orquestación IA | LangGraph             | Estado, nodos, branching y quality gate         |
| Inteligencia    | LLM Gateway           | Proveedor intercambiable y salida JSON          |
| Dominio         | Python                | Fingerprint, scoring, routing y estados         |
| Estado          | PostgreSQL            | Interacciones, análisis, activos y aprobaciones |
| Objetos         | OCI Object Storage    | Raw, analyzed, generated, approved y manifest   |
| Ejecución       | Docker Compose        | API, dashboard y base reproducibles             |

#### Componentes condicionados

- Redis y worker: se incorporan si el lote no puede resolverse de forma síncrona en menos de 30 segundos. La API devuelve run\_id y permite consultar el estado.
- n8n: opcional como webhook o conector. Debe limitarse a disparar la API, agregar metadatos y manejar notificaciones; la lógica de negocio permanece en FastAPI y LangGraph.

#### Flujo técnico

JSON o CSV -> FastAPI -> normalizador y fingerprint -> PostgreSQL -> LangGraph -> LLM Gateway -> router determinístico -> generadores -> Streamlit -> aprobación -> OCI Object Storage.

### 4 Contratos y modelo mínimo

| Entidad            | Campos esenciales                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------|
| Interaction        | id, external_id, author, channel, type, text, occurred_at, fingerprint                               |
| Analysis           | interaction_id, sentiment, topics, entities, relevance, confidence, evidence, model y prompt version |
| ContentOpportunity | analysis_id, kind, priority, reason, source_ids                                                      |
| GeneratedAsset     | opportunity_id, channel, title, body, status, version                                                |
| Approval           | asset_id, reviewer, decision, comment, timestamp                                                     |
| ObjectManifest     | object_key, content_type, sha256, etag, run_id                                                       |

## Estados del activo

DRAFT -> PENDING\_REVIEW -> APPROVED o REJECTED. PUBLISHED queda reservado para una fase posterior.

## Endpoint principal

POST /api/v1/process recibe organization\_id, community\_id, reference\_period, requested\_assets e interactions. Requiere Idempotency-Key. Devuelve run\_id, summary, analyses, opportunities, assets y storage.

## Endpoints mínimos de curaduría

- GET /api/v1/runs/{run\_id} para consultar una ejecución.
- GET /api/v1/assets?status=PENDING\_REVIEW para la cola.
- PATCH /api/v1/assets/{id} para editar.
- POST /api/v1/assets/{id}/approve y /reject para decidir.
- GET /api/v1/health para comprobar el servicio.

## 5 Pipeline de IA

1. Load: valida el lote, normaliza texto y genera fingerprint.
2. Analyze: solicita JSON estructurado con sentimiento, temas, entidades, relevancia, confianza y evidencia.
3. Route: aplica reglas de Python para crear oportunidades; el prompt no decide estados ni publicación.
4. Generate: usa prompts separados para LinkedIn y FAQ.
5. Quality: comprueba grounding, campos, privacidad y umbral de confianza.
6. Review: pausa el flujo hasta la decisión humana.
7. Persist: guarda estados en PostgreSQL y paquetes aprobados en OCI.

## Reglas iniciales

| Condición                                   | Resultado                          |
|---------------------------------------------|------------------------------------|
| Testimonio o logro y relevancia $\geq 0.75$ | Oportunidad LinkedIn               |
| Pregunta técnica útil o recurrente          | Oportunidad FAQ                    |
| Confianza $< 0.70$                          | Revisión obligatoria y advertencia |
| PII o afirmación sensible                   | Bloqueo hasta revisión             |
| Sin evidencia suficiente                    | No generar activo                  |

### Principio de seguridad

El sistema no inventa hechos. Cada afirmación del activo debe poder rastrearse a source\_interaction\_ids y evidence.

## 6 Persistencia e idempotencia

PostgreSQL es la fuente de estado operativo y auditoría. OCI Object Storage conserva objetos inmutables y paquetes; no sustituye a la base de datos.

## Layout del bucket

- raw/AAAA/MM/DD/{community\_id}/{run\_id}/input.json

- analyzed/AAAA/MM/DD/{community\_id}/{run\_id}/analysis.json
- generated/AAAA/MM/DD/{community\_id}/{run\_id}/assets.json
- approved/AAAA/MM/DD/{community\_id}/{run\_id}/package.json
- manifests/AAAA/MM/DD/{community\_id}/{run\_id}/manifest.json

### Controles obligatorios

- Fingerprint SHA 256 por interacción para evitar duplicados.
- Idempotency-Key por solicitud para que un reintento no duplique ejecuciones.
- Manifest con schema\_version, run\_id, object keys, content type, SHA 256 y fecha.
- Recibo de almacenamiento con bucket, object\_key, ETag y checksum.

## 7 Interfaz de curaduría

- Carga JSON o CSV y descarga de un archivo de ejemplo.
- Resumen con procesados, sentimiento predominante, temas y tiempo.
- Tabla de análisis con relevancia, confianza y evidencia.
- Tarjetas editables para LinkedIn y FAQ con fuentes visibles.
- Acciones Guardar edición, Aprobar y Rechazar.
- Confirmación de OCI con object key, ETag y SHA 256.

El dashboard no muestra automáticamente el payload bruto completo y nunca publica un activo. LinkedIn y FAQ requieren aprobación durante el MVP.

## 8 Calidad seguridad y observabilidad

| Área         | Control mínimo                                              |
|--------------|-------------------------------------------------------------|
| Schemas      | Pydantic versionado en todas las fronteras                  |
| LLM          | JSON válido, un reintento correctivo y fallback controlado  |
| Privacidad   | Redacción de PII y datos sintéticos en la demo              |
| Secretos     | .env local no versionado y .env.example                     |
| Trazabilidad | request_id, run_id, model_version y prompt_version          |
| Costos       | Batch pequeño, caché por fingerprint y límites de reintento |
| Logs         | JSON estructurado sin textos sensibles ni claves            |
| Gobernanza   | Registro de quién editó, aprobó o rechazó                   |

### Metas iniciales de evaluación

| Métrica                    | Meta                               |
|----------------------------|------------------------------------|
| Salidas LLM válidas        | >= 99 por ciento con fallback      |
| Afirmaciones sustentadas   | >= 95 por ciento en muestra humana |
| Precisión de oportunidades | >= 80 por ciento                   |

| Métrica                            | Meta                          |
|------------------------------------|-------------------------------|
| Tiempo de lote pequeño             | < 30 segundos según proveedor |
| Éxito de almacenamiento en pruebas | 100 por ciento en la demo     |

# 9 Pruebas y demostración

| Nivel       | Cobertura mínima                                       |
|-------------|--------------------------------------------------------|
| Unitarias   | Normalización, fingerprint, router, estados y manifest |
| Contratos   | Request y response de FastAPI y salida del LLM         |
| Integración | API, PostgreSQL, LangGraph y OCI                       |
| Interfaz    | Editar, aprobar y rechazar                             |
| End to end  | Tres casos desde carga hasta object key                |

## Dataset mínimo

20 interacciones: logros, preguntas técnicas, mensajes neutros, comentarios negativos, casos ambiguos y duplicados. La demo muestra un logro convertido en LinkedIn, una duda convertida en FAQ y un mensaje sensible bloqueado.

## Guion resumido

- Cargar el dataset y mostrar validación y deduplicación.
- Ejecutar el pipeline y revisar análisis y oportunidades.
- Editar un activo, aprobar otro y rechazar el caso sensible.
- Mostrar package.json, manifest, object key, ETag y checksum en OCI.
- Cerrar con métricas, límites y siguiente incremento.

# 10 Plan de construcción

| Fase        | Entrega                                 | Criterio de salida            |
|-------------|-----------------------------------------|-------------------------------|
| 1 Base      | Repo, Docker, health, schemas y dataset | Entorno reproducible          |
| 2 Ingesta   | JSON o CSV, fingerprint y PostgreSQL    | 20 registros sin duplicados   |
| 3 IA        | Gateway, LangGraph, análisis y router   | Salida estructurada estable   |
| 4 Contenido | LinkedIn y FAQ                          | Dos activos con evidencia     |
| 5 Curaduría | Streamlit y estados                     | Editar, aprobar y rechazar    |
| 6 OCI       | Package, manifest y adapter             | Objeto verificable en bucket  |
| 7 Cierre    | Pruebas, README y demo                  | Tres recorridos reproducibles |

## Definición de terminado

- Código revisado, pruebas críticas verdes y sin secretos.
- Prompts, schemas y modelos versionados.
- Caso feliz y caso de error documentados.
- Ejecución reproducible desde Docker Compose y README.
- Paquete final verificable en OCI Object Storage Always Free.

### Siguiente incremento

Después de validar el MVP, agregar n8n y un webhook real. Redis y worker se activan si las mediciones muestran que el procesamiento síncrono no es suficiente. Discord, Slack, publicación y OCI Compute permanecen fuera hasta estabilizar calidad y gobernanza.